CSE321 - Lab report #2

Ho Mai Phuong (20232006)
UNIST
South Korea
phuonggii@unist.ac.kr

1 INTRODUCTION

This reflective report details the systematic translation of an Entity-Relationship (ER) diagram into a formal relational database schema. The primary objective of this lab session is to bridge conceptual modeling with practical, normalized database architecture. This report explains step-by-step methodology used to achieve this, including the strict analysis of functional dependencies, the identification of normalization issues, and the logical justifications for design decisions.

2 Review of the Original Design

(1) **Staff** relation
Staff(Staff_ID, Date_Hired, Position)
Keys: {Staff_ID}
PK: {Staff_ID}
FDs: Staff_ID -> {Date_Hired, Position}
This relation was derived directly from the Staff entity in the ER diagram. I selected Staff_ID as the primary key because it serves as a unique identifier. The functional dependency assumes that a specific staff member was hired on exactly one date and holds one current position.

(2) **Laboratory** relation
- Laboratory(Lab_Name, Dept_Name, Capacity, Location, DeptNum)
  Keys: {Lab_Name, Dept_Name}
  PK: {Lab_Name, Dept_Name}
  FDs:
  {Lab_Name, Dept_Name} → {Capacity, Location, DeptNum}
  Lab_Name → {Capacity, Location}
  Dept_Name → DeptNum
  Derived from the ER diagram of Laboratory entity. I set composite primary key {Lab_Name, Dept_Name}. I identified partial functional dependencies because attributes like Capacity and Location can be decided with lab names in real life, similarly DeptName can decide DeptNum.
- Department(Dept_Name, DeptNum)
- L1(Lab_Name, Capacity)
- L3(Lab_Name, Location)
- L4(Lab_Name, Dept_Name)

(3) **Equipment** relation
Equipment(ID, Date_Purchased, Name, Model_No)
Keys: {ID}
PK: {ID}
FDs: ID → {Date_Purchased, Name, Model_No}
According to the ER diagram. ID was used as the primary key as it uniquely determines all other non-prime attributes.

(4) **Student** relation
Student(Student_ID, year, majors, minors)
Keys: {Student_ID}
PK: {Student_ID}
FDs: Student_ID → {majors, minors, year}

(5) **Course** relation
Course(Cid, Credit, Name, Date_ time)
Keys: {Cid}
PK: {Cid}
FDs:

Cid → { Credit, Name, Date_ time}
Name → {Credit, Date_Time}

According to the ER diagram, Cid is the only primary key. Moreover, I assume that we can know course credits and their schedules only by their names.

(6) **Professor** relation
Professor(Professor_ID, Dept_Name, Major)
Keys: {Professor_ID}
PK: {Professor_ID}
FDs: Professor_ID -> {Dept_Name, Major}
According to the ER diagram. Professor_ID was used as the primary key. I also assume other attributes don't depend on each other.

(7) **Person** relation
- Persons(Person_ID, Name, Schools, Phone, Email, Address, Nation, PostalCode)
  Keys: {Person_ID}
  PK: {Person_ID}
  FDs:
  Person_ID → {Name, Phone, Schools, Email, Address, Nation, PostalCode}
  Address → {Nation, PostalCode}
  According to the ER diagram. Person_ID was used as the primary key and it determines all attributes. Realistically, I assume that by knowing a person's address, we may know their nations and postal code as well.
- PerAddr(Address, Nation, PostalCode)
- P2(Person_ID, Name, Phone, Schools, Email, Address)

(8) **Comment_Suggestion** relation
Comment_Suggestion(Date, Topic)
Keys: {Date}, {Topic}
PK: {Date}
FD: *None*
During the in-class activity, I assume there are 2 keys of this relation by looking at the ER diagram. Date and Topic are not dependent in real life so there is no FD according to my assumption.

3 Revised Design

(1) **Person** relation

- PersonSchool(Person_ID, Schools)
  Keys: {Person_ID, Schools}
  PK: {Person_ID, Schools}
  FDs: *None*
  Relation *PersonSchool* is in 3NF because it utilizes a composite primary key consisting of both attributes Person_ID, School.
- Persons(Person_ID, Name, Phone, Email, Address, Nation, PostalCode)
  Keys: {Person_ID}
  PK: {Person_ID}
  FDs:
  Person_ID → {Name, Phone, Email, Address, Nation, PostalCode}
  Address → {Nation, PostalCode}
  Relation *Persons* is **not** in 3NF because Address → {Nation, PostalCode} is a violation (Address is not a key).
  Therefore we need further decomposition.
- PerAddr(Address, Nation, PostalCode)
  Keys: {Address}
  PK: {Address}
  FDs: Address → {Nation, PostalCode}
  The relation *PerAddr* is in 3NF because the above FD is OK (Address is a key of PerAddr).
- P2(Person_ID, Name, Phone, Email, Address)
  Keys: {Person_ID}
  PK: {Person_ID}
  FDs: Person_ID → {Name, Phone, Email, Address}
  Relation *P2* is in 3NF because the above FD is OK (Person_ID is a key of P2)

(2) **Laboratory** relation

- Laboratory(Lab_Name, Dept_Name, Capacity, Location, DeptNum)
  Keys: {Lab_Name, Dept_Name}
  PK: {Lab_Name, Dept_Name}
  FDs:
  {Lab_Name, Dept_Name} → {Capacity, Location, DeptNum}
  Lab_Name → {Capacity, Location}
  Dept_Name → DeptNum
  Relation *Laboratory* is **not** in 3NF because Dept_Name → DeptNum is a violation (Dept_Name is not a key of Laboratory).
  Therefore we need further decomposition.
- Department(Dept_Name, DeptNum)
  Keys: {Dept_Name}
  PK: {Dept_Name}
  FD: Dept_Name → DeptNum
  Relation *Department* is in 3NF because the above FD is okay (Dept_Name is a key of Department)
- Lab(Lab_Name, Dept_Name, Capacity, Location)
  Keys: {Lab_Name, Dept_Name}
  PK: {Lab_Name, Dept_Name}
  FDs: Lab_Name → {Capacity, Location}
  Relation *Lab* is **not** in 3NF because the above FD is a violation (Lab_Name is not a key of Lab)
  So we further decompose.
- L1(Lab_Name, Capacity, Location)
  Keys: {Lab_Name}
  PK: {Lab_Name}
  FDs: Lab_Name → {Capacity, Location}
  Relation *L1* is in 3NF because the above FD is OK (Lab_Name is a key of L1)
- L2(Lab_Name, Dept_Name)
  Keys: {Lab_Name, Dept_Name}
  PK: {Lab_Name, Dept_Name}
  FDs: *None*
  Relation *L2* is in 3NF because its attribute is a composite primary key.

(3) **Equipment** relation

Equipment(Lab_Name, Dept_Name, ID, Date_Purchased, Name, Model_No)
Keys: {Lab_Name, Dept_Name, ID}
PK: {Lab_Name, Dept_Name, ID}
FDs: {Lab_Name, Dept_Name, ID} → {Date_Purchased, Name, Model_No}
Relation *Equipment* is in 3NF because the above FD is OK ({Lab_Name, Dept_Name, ID} is a key of Equipment).

(4) **Student** relation

- Student(Person_ID, Student_ID, year)
  Keys: {Person_ID}, {Student_ID}
  PK: {Student_ID}
  FDs:
  Student_ID → {Person_ID, year}
  Person_ID → {Student_ID, year}
  Relation *Student* is in 3NF because:
  – Student_ID → {Person_ID, year} is OK, since Student_ID is a key of Student.
  – Person_ID → {Student_ID, year} is OK, since Person_ID is a key of Student.
- Student_Majors(Student_ID, Majors)
  Keys: {Student_ID, Majors}
  PK: {Student_ID, Majors}
  FDs: *None*
  Relation *Student_Majors* is in 3NF because its attribute is a composite primary key.
- Student_Minors(Student_ID, Minors)
  Keys: {Student_ID, Minor}
  PK: {Student_ID, Minors}
  FDs: *None*
  Relation *Student_Minors* is in 3NF because its attribute is a composite primary key.

(5) **Course** relation

Course(Cid, Credit, Name, Date_ time)
Keys: {Cid}
PK: {Cid}
FDs: Cid → { Credit, Name, Date_ time}
Relation *Couse* is in 3NF because the above FD is OK (Cid is a key of Course)

(6) **Professor** relation

Professor(Person_ID, Professor_ID, Dept_Name, Major)
Keys: {Person_ID}, {Professor_ID}
PK: {Professor_ID}
FDs:

Professor_ID -> {Dept_Name, Major, Person_ID},
Person_ID -> {Dept_Name, Major, Professor_ID}

Relation *Professor* is in 3NF because:
- Professor_ID -> {Dept_Name, Major, Person_ID} is OK, since Professor_ID is a key of Professor.
- Person_ID -> {Dept_Name, Major, Professor_ID} is OK, since Person_ID is a key of Professor.

(7) **Staff** relation
Staff(Person_ID, Staff_ID, Date_Hired, Position)
Keys: {Person_ID}, {Staff_ID}
PK: {Staff_ID}
FDs:

Staff_ID -> {Date_Hired, Position, Person_ID},
Person_ID -> {Date_Hired, Position, Staff_ID}

Relation *Staff* is in 3NF because:
- Staff_ID -> {Date_Hired, Position, Person_ID} is OK, since Staff_ID is a key of Staff.
- Person_ID -> {Date_Hired, Position, Staff_ID} is OK, since Person_ID is a key of Staff.

(8) **Stakeholder** relation
Stakeholder(Person_ID, Domain)
Keys: {Person_ID}
PK: {Person_ID}
FDs: Person_ID -> Domain
Relation *Stakeholder* is in 3NF because the above FD is OK, since Person_ID is a key of Stakeholder.

(9) **Comment_Suggestion** relation
Comment_Suggestion(Person_ID, Date, Topic)
Keys: {Person_ID, Date, Topic}
PK: {Person_ID, Date, Topic}
FD: *None*
Relation *Comment_Suggestion* is in 3NF because its attribute is a composite PK.

(10) **Many-to-Many** relationships
- Take(Student_ID, Cid)
  Keys: {Student_ID, Cid}
  PK: {Student_ID, Cid}
  FDs: *None*
  Relation *Take* is in 3NF because its attribute is a composite PK.
- Assign(Student_ID, Lab_Name, Dept_Name)
  Keys: {Student_ID, Lab_Name, Dept_Name}
  PK: {Student_ID, Lab_Name, Dept_Name}
  FDs: *None*
  Relation *Assign* is in 3NF because its attribute is a composite PK.
- Experiments(Student_ID, Lab_Name, Dept_Name, Date)
  Keys: {Student_ID, Lab_Name, Dept_Name}
  PK: {Student_ID, Lab_Name, Dept_Name}
  FDs: {Student_ID, Lab_Name, Dept_Name} → Date
  Relation *Experiments* is in 3NF because the above FD is OK, since {Student_ID, Lab_Name, Dept_Name} is a key of Experiments.

4 Revision Details

Revision 1: Inheriting the primary key of the superclass Person:

As stated in the second section, I originally included only attributes for the specific relations. However, *Student, Staff, Professor, Stakeholder* are subclasses of *Person*. Therefore I added *Person_ID* to those relations as they must inherit the superclass's primary key to accurately reflect inheritance structure.

- Staff(Person_ID, Staff_ID, Date_Hired, Position)
  Keys: {Person_ID}, {Staff_ID}
  PK: {Staff_ID}
  FDs:

  Staff_ID -> {Date_Hired, Position, Person_ID},
  Person_ID -> {Date_Hired, Position, Staff_ID}

- Student(Person_ID, Student_ID, Majors, Minors, Year)
- Professor(Person_ID, Professor_ID, Dept_Name, Major)
  Keys: {Person_ID}, {Professor_ID}
  PK: {Professor_ID}
  FDs:

  Professor_ID -> {Dept_Name, Major, Person_ID},
  Person_ID -> {Dept_Name, Major, Professor_ID}

- Stakeholder(Person_ID, Domain)
  Keys: {Person_ID}
  PK: {Person_ID}
  FDs: Person_ID -> Domain
- Comment_Suggestion(Person_ID, Date, Topic)
  Keys: {Person_ID, Date, Topic}
  PK: {Person_ID, Date, Topic}
  FD: *None*

Revision 2: Decomposing Multi-valued Attributes (Achieving 1NF)

As stated in the ER diagram, the *Person* entity contains the multi-valued attribute *Schools*, and the *Student* entity contains *Majors* and *Minors*. Storing multiple values or lists within a single attribute violates First Normal Form (1NF). To resolve this and move the schema toward 3NF, these attributes were extracted into their own separate tables.

- Person_Schools(Person_ID, School)
  Keys: {Person_ID, School}
  PK: {Person_ID, School}
  FDs: *None*
- Student_Majors(Student_ID, Majors)
  Keys: {Student_ID, Majors}
  PK: {Student_ID, Majors}
  FDs: *None*
- Student_Minors(Student_ID, Minors)
  Keys: {Student_ID, Minors}
  PK: {Student_ID, Minors}
  FDs: *None*

*Note: Following this decomposition, the base schemas for **Person** and **Student** no longer contain multi-valued attributes. This allows the base tables to fully satisfy 1NF, 2NF, and 3NF.*

Revision 3: Mapping Many-to-Many (M:N) Relationships

In this revision, I created a new junction table for each M:N relationship. The primary key of this new table is a composite key made up of the primary keys of the participating entities.

- Take(Student_ID, Cid)
  Maps the M:N relationship between Student and Course.
  Keys: {Student_ID, Cid}
  PK: {Student_ID, Cid}
  FDs: *None*
- Assign(Student_ID, Lab_Name, Dept_Name)
  Maps the M:N relationship between Graduate students and Research Laboratory. Laboratory utilizes a composite primary key, and Research Laboratory is its subclass. Similarly, Grad is a subclass of Student.
  Keys: {Student_ID, Lab_Name, Dept_Name}
  PK: {Student_ID, Lab_Name, Dept_Name}
  FDs: *None*
- Experiments(Student_ID, Lab_Name, Dept_Name, Date)
  Maps the M:N relationship between Undergrad students and Teaching Laboratories. Undergrad is a subclass of Student and Teaching Laboratory is a subclass of Laboratory, so they inherit their superclass's PK.
  Keys: {Student_ID, Lab_Name, Dept_Name}
  PK: {Student_ID, Lab_Name, Dept_Name}
  FDs: {Student_ID, Lab_Name, Dept_Name} → Date

Revision 4: Mapping Weak Entities

In the Entity-Relationship model, a weak entity is one that cannot uniquely identify using only its own local attributes. To resolve this and satisfy entity integrity, a weak entity must absorb the primary key of its identifying owner entity.

- Equipment(Lab_Name, Dept_Name, ID, Name, Date_Purchased, Model_No)
  Equipment absorbs the composite primary key {Lab_Name, Dept_Name} from its owner, Laboratory, and combines it with its own partial key, ID.
  Keys: {Lab_Name, Dept_Name, ID}
  PK: {Lab_Name, Dept_Name, ID}

  FDs: {Lab_Name, Dept_Name, Equipment_ID} → {Name, Date_Purchased, Model_No}
- Comment_Suggestion(Person_ID, Date, Topic)
  It absorbs the primary key {Person_ID} from its owner, Stakeholder. Because Stakeholder inherits its primary key from the Person superclass, Person_ID is the correct identifying attribute to cascade down into this weak entity's composite primary key.
  Keys: {Person_ID, Date, Topic}
  PK: {Person_ID, Date, Topic}
  FDs: *None*

5 CONCLUSION

This lab session provides hands-on experience in translating a conceptual ER diagram into a structurally sound relational database. Throughout this process, I learned how to identify and resolve design flaws using normalization techniques of Third Normal Form (3NF). Specifically, extracting multi-valued attributes, accurately mapping subclass inheritance, and creating junction tables for Many-to-Many relationships reinforce the importance of functional dependencies in eliminating data redundancy and maintaining database integrity.